

第8章 软件包管理





第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. APT工具
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式

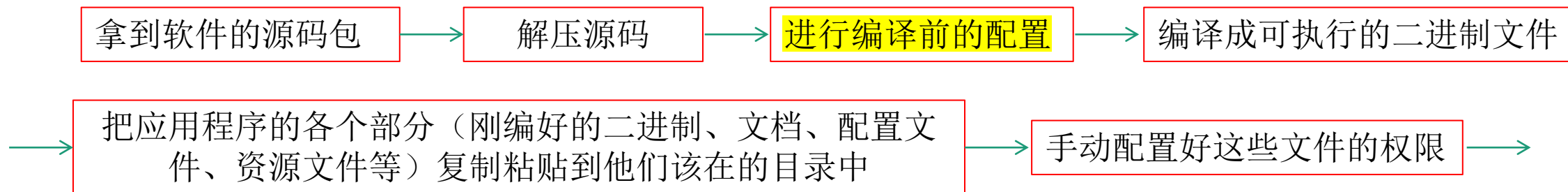


第8章 软件包管理

1) 从源代码安装软件

早期Linux主要使用源代码包发布软件。

直接将源代码编译成二进制文件，并对系统进行相关配置。



题目

在Linux下使用源代码安装软件的基本步骤：首先，使用_____命令将软件包的归档文件解压缩到一个目录下，然后使用_____命令对源代码进行配置，再则使用_____命令进行编译，最后_____命令进行安装。



第8章 软件包管理

2) 使用软件包安装软件

软件包将应用程序的二进制文件、配置文档和帮助文档等合并打包在一个文件中。**主流的软件包格式有两种：RPM和Deb。**

下载.deb文件



赋予执行权限



安装软件包： `sudo dpkg -i /path/to/your_file.deb`

Deb包的组成有三个主要部分：

控制文件：软件的元数据，如软件名称、版本号、作者、依赖关系等等。

数据文件：软件的二进制程序、库文件、配置文件、文档等等。

脚本文件：软件安装和卸载过程中需要执行的脚本命令。

```
1 | myapp_1.0.0-1/  
2 |   └─ DEBIAN/  
3 |       └─ control  
4 |   └─ usr/  
5 |       └─ bin/  
6 |           └─ myapp  
7 |       └─ share/  
8 |           └─ doc/  
9 |               └─ myapp/  
10 |                   └─ README
```



第8章 软件包管理

3) 高级软件包管理工具

Yum: 基于RPM包的软件包管理器。

APT: Debian及其派生发行版（如Ubuntu）的软件包管理器。

Snap: 一种全新的软件包安装管理方式，可以不依赖于第三方系统功能库独立包装，代表未来软件包安装的发展方向。

```
root@ -virtual-machine:~# apt-get install nmap
```

```
正在读取软件包列表... 完成
```

```
正在分析软件包的依赖关系树... 完成
```

```
正在读取状态信息... 完成
```

```
无 apt 软件包 "nmap"，但有一个同名的 snap 软件包
```

```
尝试 "snap install nmap"
```

```
E: 无法定位软件包 nmap
```

CSDN @花白默示



第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. APT工具
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式



第8章 软件包管理

- 查看Deb软件包——**dpkg (debian package)**

dpkg是为 Debian 专门开发的软件管理系统，方便软件的安装、更新及移除。

- 使用选项-l列出软件包的简要信息：

```
dpkg -l <.deb file name>
```

- 使用选项-S获知已安装的软件包拥有哪些文件：

```
dpkg -S <.deb file name>
```

Ubuntu系统不推荐使用deb软件包直接安装，因为要解决软件包依赖问题，安装也比较麻烦。



第8章 软件包管理

- 安装Deb软件包

- 所有的软件包安装之前必须保证所依赖的库和依赖软件已经安装到系统上。
- 首先要获取Deb软件包文件，然后使用选项-i安装Deb软件包

```
sudo dpkg -i <.deb file name>
```

- 卸载Deb软件包

- 使用选项-r卸载软件包同时会保留该软件的配置信息：

```
sudo dpkg -r <.deb file name>
```

- 使用dpkg工具卸载软件包不会自动解决依赖性问题。



第8章 软件包管理

• 应用场景——没有互联网连接的场景

提前先准备好离线安装包，拷贝到这些离线安装场景的设备上进行安装

Step1. 查看软件依赖包列表——`sudo apt-cache depends <package_name>`



Step2. 软件安装包下载——`sudo apt-get download <package_name>`



Step3. 软件依赖包递归下载

```
for i in $(apt-cache depends iperf3 | grep -E 'Depends|Recommends|Suggests' | cut -d ':' -f 2,3 | sed -e s/'</'/' -  
e s/'>/'/'/); do sudo apt-get download $i 2>>errors.txt; done
```



Step4. 软件包的打包压缩 和 解压缩（tar.gz）——`tar -zcvf tar-name.tar.gz file-names`



Step5. 安装软件包——`sudo dpkg -i *.deb`——一次性安装所有的.deb安装包



第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. **APT工具**
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式



第8章 软件包管理

• 理解APT

1) 基本功能

- 从Internet上的软件源**下载**最新的软件包元数据、二进制包或源代码包。
- 利用下载到本地的软件包元数据，完成软件包的搜索和系统的**更新**。
- **安装和卸载**软件包时自动寻找最新版本，并**自动解决软件的依赖关系**。

2) 软件源

- Ubuntu安装时已提供Ubuntu官方的网络安装来源。
- 可以使用系统安装光盘作为安装源，或从非官方的软件源中下载非官方的软件。
- /var/lib/apt/lists目录存放的是已经下载的各软件源的元数据（电子式目录）。



第8章 软件包管理

- apt常用命令

apt命令	被替代的命令	功能说明
apt update	apt-get update	获取最新的软件包列表，同步/etc/apt/sources.list和/etc/apt/sources.list.d中列出的源的索引，以确保用户能够获取最新的软件包
apt upgrade	apt-get upgrade	更新当前系统中所有已安装的软件包，并同时所更新的软件包相关的所依赖的软件包
apt install	apt-get install	下载、安装软件包并自动解决依赖关系
apt remove	apt-get remove	卸载指定的软件包
apt autoremove	apt-get autoremove	自动卸载所有未使用的软件包
apt purge	apt-get purge	卸载指定的软件包及其配置文件
apt full-upgrade	apt-get dist-upgrade	在升级软件包时自动处理依赖关系



第8章 软件包管理

- apt常用命令

apt命令	被替代的命令	功能说明
apt source	apt-get source	下载软件包的源代码
apt clean	apt-get clean	清理已下载的软件包，实际上是清除/var/cache/apt/archives 目录中的软件包，不会影响软件的正常使用
apt autoclean	apt-get autoclean	删除已卸载的软件的软件包备份
apt list	无	列出包含条件的软件包（已安装、可升级等）
apt search	apt-cache search	搜索应用程序
apt show	apt-cache show	显示软件包详细信息
apt edit-sources	无	编辑软件源列表



第8章 软件包管理

• **sudo apt-get update** 执行这条命令后计算机做了什么？

清华源

```
1 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
2 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
3 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
4 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
5 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
6 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
7 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
8 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
```

换源：——/etc/apt/sources.list → 告知软件源镜像站点的地址

→ 所指向的镜像站点所具有的软件资源清单 → 本地主机知晓可以申请哪些资源

→ “apt-get update” 命令刷新软件源，建立更新软件包列表 → /var/lib/apt/lists/目录



第8章 软件包管理

• **sudo apt-get install XXX** 执行这条命令后计算机做了什么？

使用“apt-get install”下载软件包大体分为4步：

第一步，扫描本地存放的软件包更新列表（由“apt-get update”命令刷新更新列表，也就是/var/lib/apt/lists/），找到最新版本的软件包；



第二步，进行软件包依赖关系检查，找到支持该软件正常运行的所有软件包；



第三步，从软件源所指的镜像站点中，下载相关软件包；



第四步，解压软件包，并自动完成应用程序的安装和配置。



第8章 软件包管理

- **sudo apt-get upgrade**

执行这条命令后计算机做了什么？

一条命令就可以轻松地将系统中的所有软件包一次性升级到最新版本！

第一步，需要修改软件源配置文件



第二步，使用“apt-get update”命令刷新软件包更新列表。



第三步，使用“apt-get dist-upgrade”命令为系统升级新的版本。

Ubuntu20.10 (groovy)

阿里源：

```
1 | deb http://mirrors.aliyun.com/ubuntu/ groovy main restricted universe multiverse
```

Ubuntu20.04 (focal)

阿里源：

```
1 | deb http://mirrors.aliyun.com/ubuntu/ focal main restricted universe multiverse
```

Ubuntu19.10 (eoan)

阿里源：

```
1 | deb http://mirrors.aliyun.com/ubuntu/ eoan main restricted universe multiverse
```




第8章 软件包管理

- 几个和apt-get相关的目录:

`/var/lib/dpkg/available`

文件的内容是软件包的描述信息，包括当前系统所使用的 **ubuntu** 安装源中的所有软件包。

`/var/cache/apt/archives`

目录是在用 **apt-get install** 安装软件时，软件包的临时存放路径

`/etc/apt/sources.list`

存放的是软件源站点

`/var/lib/apt/lists`

使用**apt-get update**命令会从**/etc/apt/sources.list**中下载软件列表，并保存到该目录



第8章 软件包管理

- **卸载软件包：**

- `apt remove`——卸载一个已安装的软件包，但会保留该软件包的配置文件。
- 要同时删除配置文件，则要执行`apt purge`命令。
- 更彻底的删除，可执行以下命令删除与该软件包及其所依赖的、**不再使用**的软件包：

```
sudo apt-get autoremove 软件包名
```



第8章 软件包管理

- **apt其他子命令介绍:**

- **list**——列出软件包。
- **search**——查找使用参数定义的软件包并列出该软件包的相关信息。
- **show**——可以查看指定名称的软件包的详细信息。
- **depends**——可以查看软件包所依赖的软件包。
- **rdepends**——可以查看软件包被哪些软件包所依赖:
- **policy**——显示软件包的安装状态和版本信息。



第8章 软件包管理

- **apt无法完成:**

- 与特定软件包相关的所有文件——

```
dpkg -L <软件包名>
```

- 特定文件属于哪个包——

```
dpkg --search <文件名>
```



在 Ubuntu 中，dpkg 和 apt 是软件包管理的核心工具，但二者的角色和功能有显著区别：

特性	**dpkg**	**apt**
层级	底层工具（直接操作 .deb 文件）	高层工具（管理软件仓库和依赖关系）
依赖处理	✗ 不自动解决依赖	✓ 自动解析并安装依赖
数据源	仅限本地 .deb 文件	远程仓库（通过 sources.list 配置）
典型用途	手动安装离线包、低级维护	日常安装/更新/删除软件包



第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. APT工具
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式



第8章 软件包管理

- **目前软件管理中的问题：**

- 1) 程序文件遍布整个系统
- 2) 需要复杂的代码来管理安装与更新
- 3) 操作不是事务性的（如果操作失败，对系统的影响仍然会保留）



Snap包安装和管理



第8章 软件包管理

- 使用Snap包安装软件

- 搜索要安装的Snap包

`snap find <要搜索的文本>`

- 查看Snap包的详细信息

`snap info Snap 包名`

- 搜索要安装的Snap包

`snap install Snap 包名`

- 列出已经安装的Snap包

`snap list`

- 搜索要安装的Snap包

`snap find <要搜索的文本>`

- 查看Snap包的详细信息

`snap info Snap 包名`

- 搜索要安装的Snap包

`snap install Snap 包名`



第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. APT工具
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式



第8章 软件包管理

源代码安装的基本步骤：

01
STEP

下载和解压软件包。

02
STEP

执行configure生成编译配置文件Makefile。

03
STEP

执行make命令编译源代码。

04
STEP

执行make install安装软件。



第8章 软件包管理

1. Linux软件包管理的发展过程
2. Deb软件包管理
3. APT工具
4. Snap包安装和管理
5. 使用源代码安装
6. 其他安装方式



第8章 软件包管理

- 使用新立得软件包管理器

```
sudo apt-get install synaptic
```

